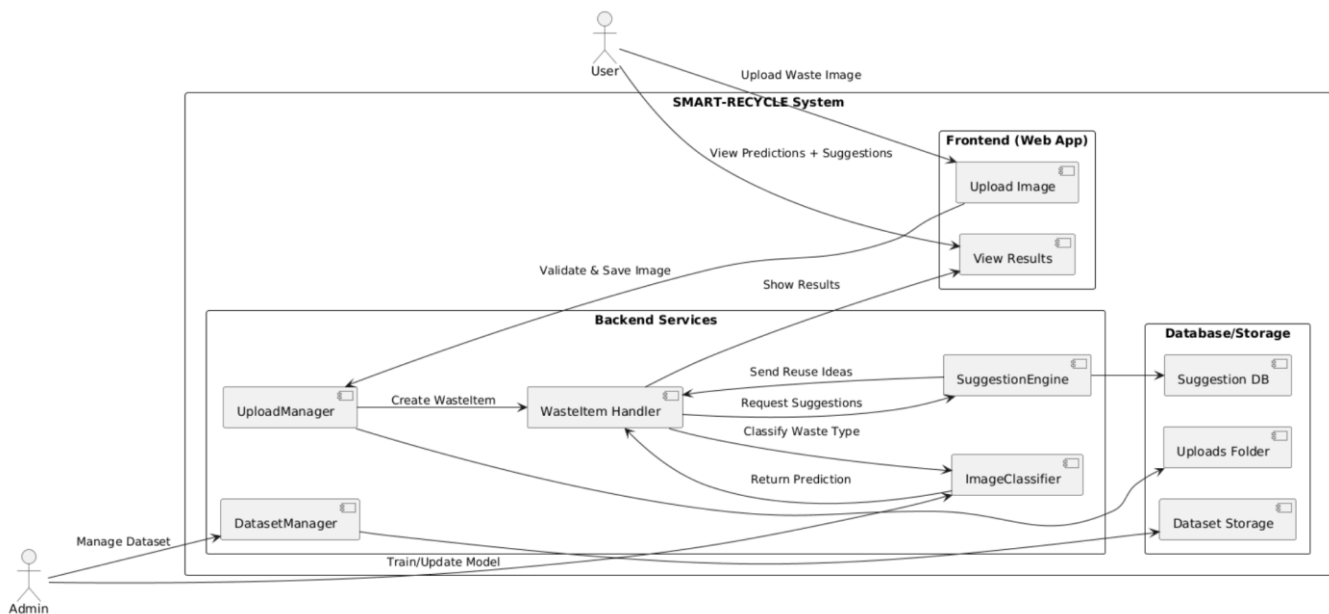


Project Title

AI-Powered Waste Sorting and Suggestion System (SMART-RECYCLE)



Project Objective [For any other reference => Link to ChatGpt](#)

Create a system that classifies waste materials into categories like **plastic, paper, metal, cardboard, glass, and trash** using **image processing** and **convolutional neural networks (CNNs)**, thereby enhancing recycling efficiency.

The system will also provide **recommendations and suggestions** for reusing/upcycling the detected waste into useful products.

System Overview

- User uploads an image of waste material.
- System preprocesses the image → CNN model predicts the category.
- Based on the predicted class, a **Suggestion Engine** provides creative reuse ideas, tutorials, and safety guidelines.
- Users can provide **feedback** (helpful/not helpful, comments).
- Admin can **manage dataset, retrain model, and update suggestions**.

Project Breakdown – Step-by-Step

1. Dataset Collection

Waste categories: plastic, paper, metal, cardboard, glass, trash

References:

- GitHub: <https://github.com/garythung/trashnet>
- HuggingFace: <https://huggingface.co/datasets/garythung/trashnet>
- Kaggle_1: <https://www.kaggle.com/datasets/asdasdasdasdas/garbage-classification>
- Kaggle_2: <https://www.kaggle.com/datasets/kneroma/tacotrashdataset>
- Kaggle_3: <https://www.kaggle.com/datasets/techsash/waste-classification-data>

- Google Drive (TrashNet mirror): https://drive.google.com/drive/folders/1CTvT_gnTvwlcKwJ8yz4jUOs0JYTKrplA
- Research Paper: <https://cs229.stanford.edu/proj2016/report/ThungYang-ClassificationOfTrashForRecyclabilityStatus-report.pdf>

2. Data Preprocessing

- Resize images (128×128 or 150×150)
- Labeling (plastic, paper, metal, cardboard, glass, trash)
- Augmentation: rotation, flip, zoom, brightness jitter
- Normalize pixel values (0–255 → 0–1)
- Train/Validation/Test split: **70:15:15**
- Store metadata: {filename, label, source, resolution, date}

3. Model Architecture

- **Baseline:** Custom CNN (Conv2D + MaxPool + Dense)
- **Optimized:** Transfer learning with MobileNetV2 / ResNet50 (fine-tuned)
- **Loss Function:** categorical_crossentropy
- **Optimizer:** Adam (lr=0.001)
- **Metrics:** Accuracy, Precision, Recall, F1-score

4. Model Evaluation

- Accuracy
- Confusion Matrix
- Per-class Precision/Recall
- Error Analysis
- Model versioning with metadata file (model_meta.json)

5. Recommendation & Suggestion Engine

- **Rule-based (JSON mapping):**
Example:
 - {
 - "plastic": ["Make a plant pot", "DIY bird feeder", "Desk organizer"]
 - }
- **Tutorial links** for each suggestion (YouTube/Instructables).
- **Safety notes** (e.g., wash bottles, avoid rust).
- **Future upgrade:** ML-based personalized recommender.

6. Feedback Loop

- Users can rate suggestions (👍 / 👎) and add comments.
- Feedback stored in DB (feedback collection).
- Admin can retrain model and refine suggestions.

7. Frontend + Backend Integration

- **Frontend:** HTML, CSS, JavaScript
- **Backend:** Flask (or Node.js + Express)

- **Model Serving:** Flask → tensorflow.keras.models.load_model
 - **Endpoints:**
 - POST /api/upload → Upload image
 - GET /api/predict/<upload_id> → Get predicted class + confidence
 - GET /api/suggestions/<class> → Fetch reuse ideas
 - POST /api/feedback → Store feedback
 - POST /admin/train → Retrain model (admin only)
-

8. Deployment

- Containerized with Docker + docker-compose
 - Hosted on **Render / AWS / Heroku**
 - Web server: Nginx + Gunicorn
 - Optional: TensorFlow-Serving for scalable inference
-

9. Security & Privacy

- File type validation (JPEG/PNG only)
 - Max upload size limit
 - HTTPS enforced
 - Admin endpoints protected with JWT authentication
 - Option to auto-delete user uploads after processing
-

10. Non-Functional Requirements

- **Performance:** Prediction latency < 2s
 - **Scalability:** Support 100+ concurrent users (Docker + autoscaling)
 - **Usability:** Simple, mobile-responsive UI
 - **Reliability:** Graceful fallback if model/API unavailable
-

11. Testing Plan

- **Unit Tests:** preprocessing, API routes, suggestion fetcher
 - **Integration Tests:** upload → predict → suggestions → feedback flow
 - **Model Tests:** Ensure accuracy doesn't drop after retraining
 - **User Testing:** Real users test suggestions usability
-

12. Admin Features

- **Dataset Manager:** Upload/remove images, correct labels
 - **Model Trainer:** Trigger retraining, view logs
 - **Suggestions Manager:** Add/edit/delete ideas
 - **Analytics Dashboard:** Usage metrics, top suggestions, misclassifications
-

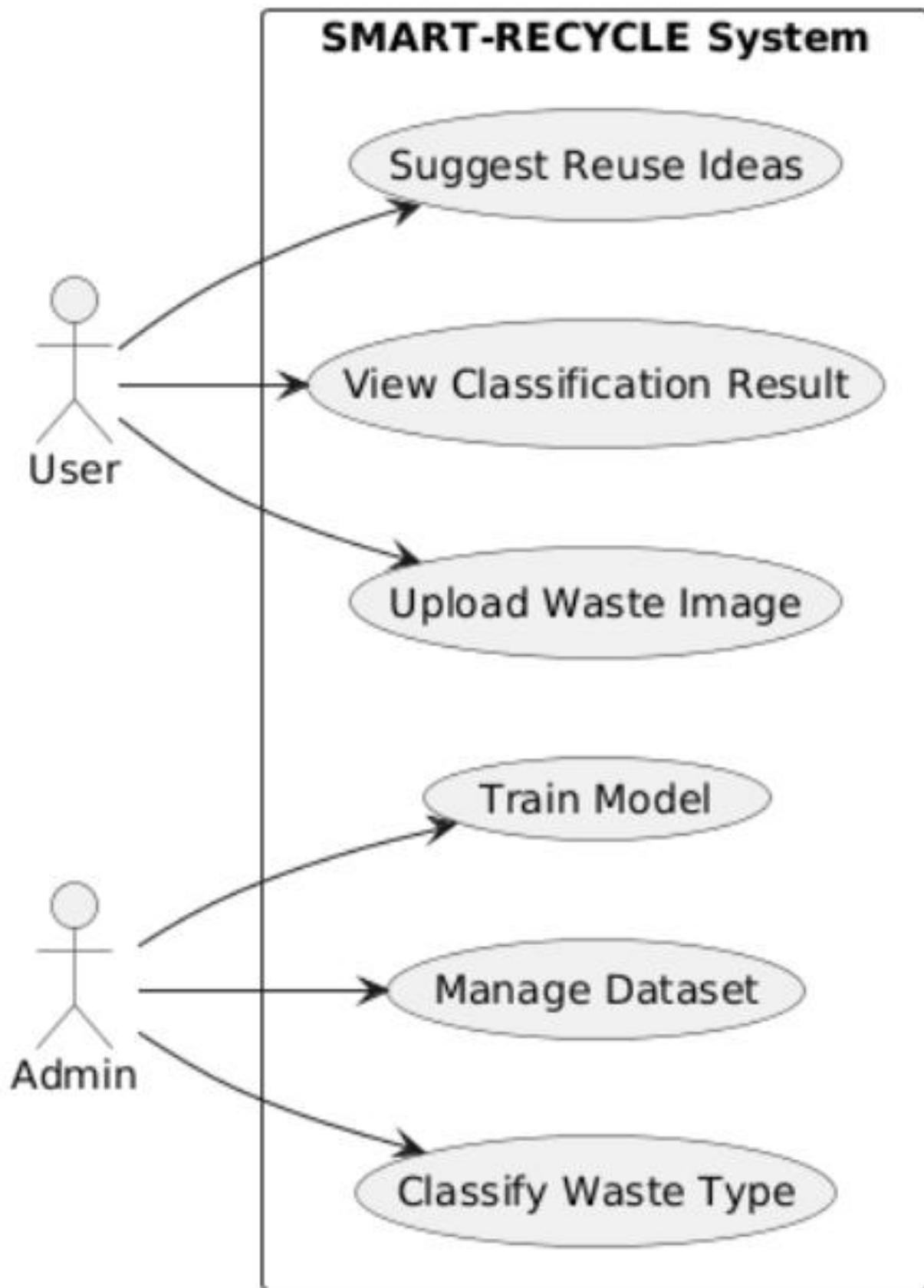
13. Future Enhancements

- Real-time video waste classification
- Mobile app (React Native / Flutter)
- Integration with **IoT Smart Bins**
- Marketplace for upcycled products
- GPT-powered suggestion engine for creative ideas

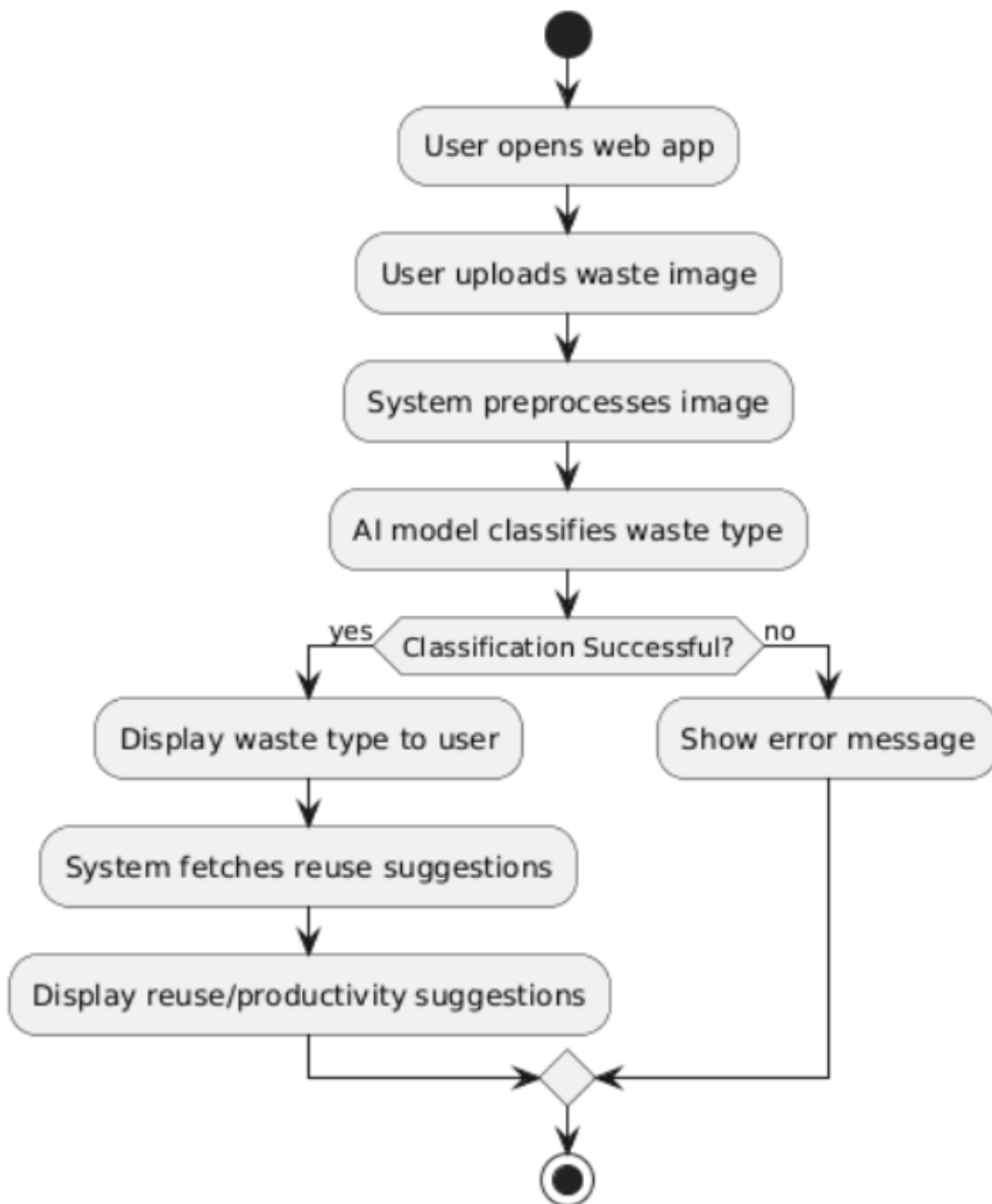
Short Flow (Summary)

- **Image Processing:** Python, TensorFlow, NumPy
- **Model Training:** TensorFlow/Keras
- **Web App:** Flask + HTML/CSS/JS
- **Suggestion Engine:** Rule-based JSON mapping
- **Deployment:** Docker + Render/AWS

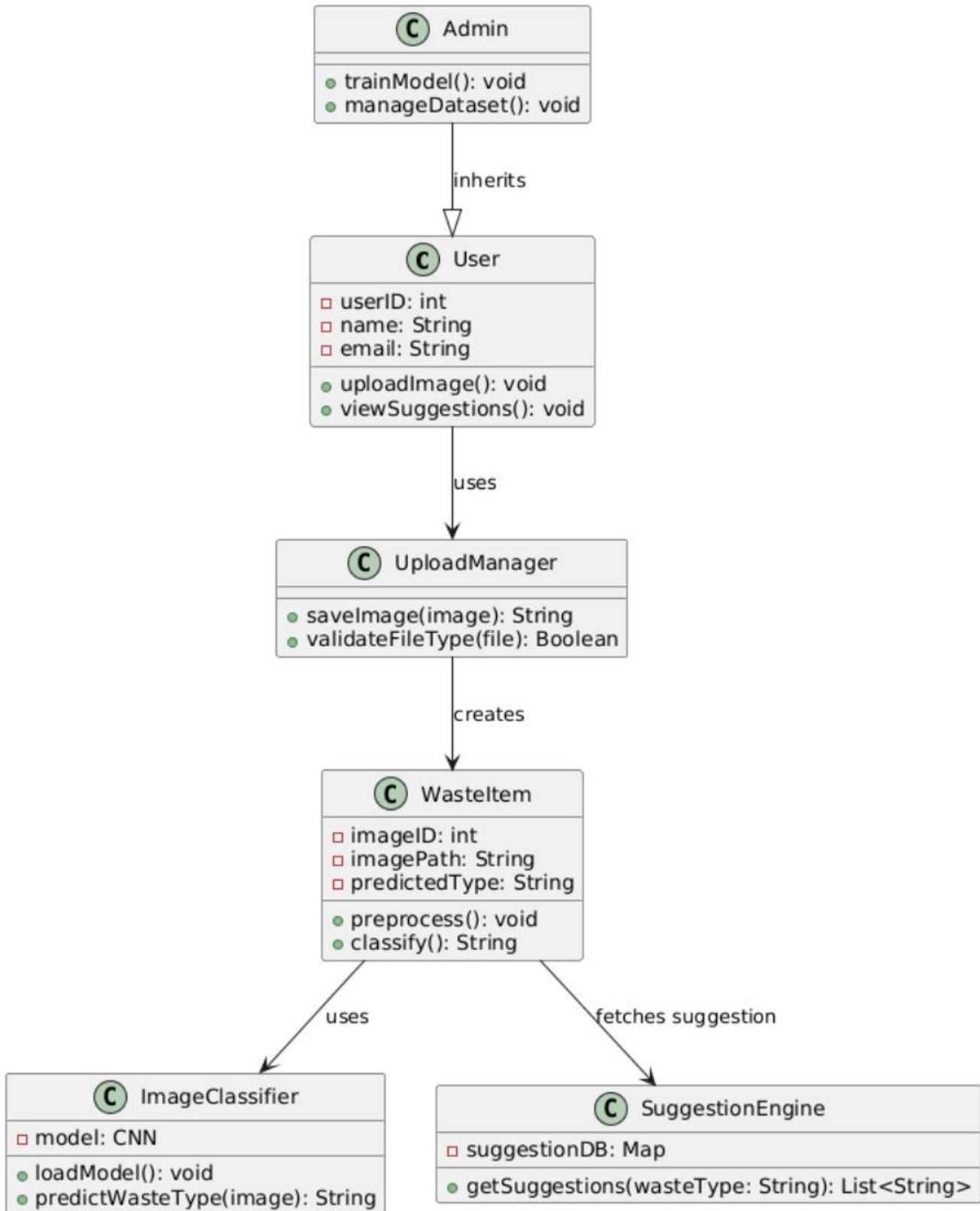
USE Case diagram:



Activity Diagram:

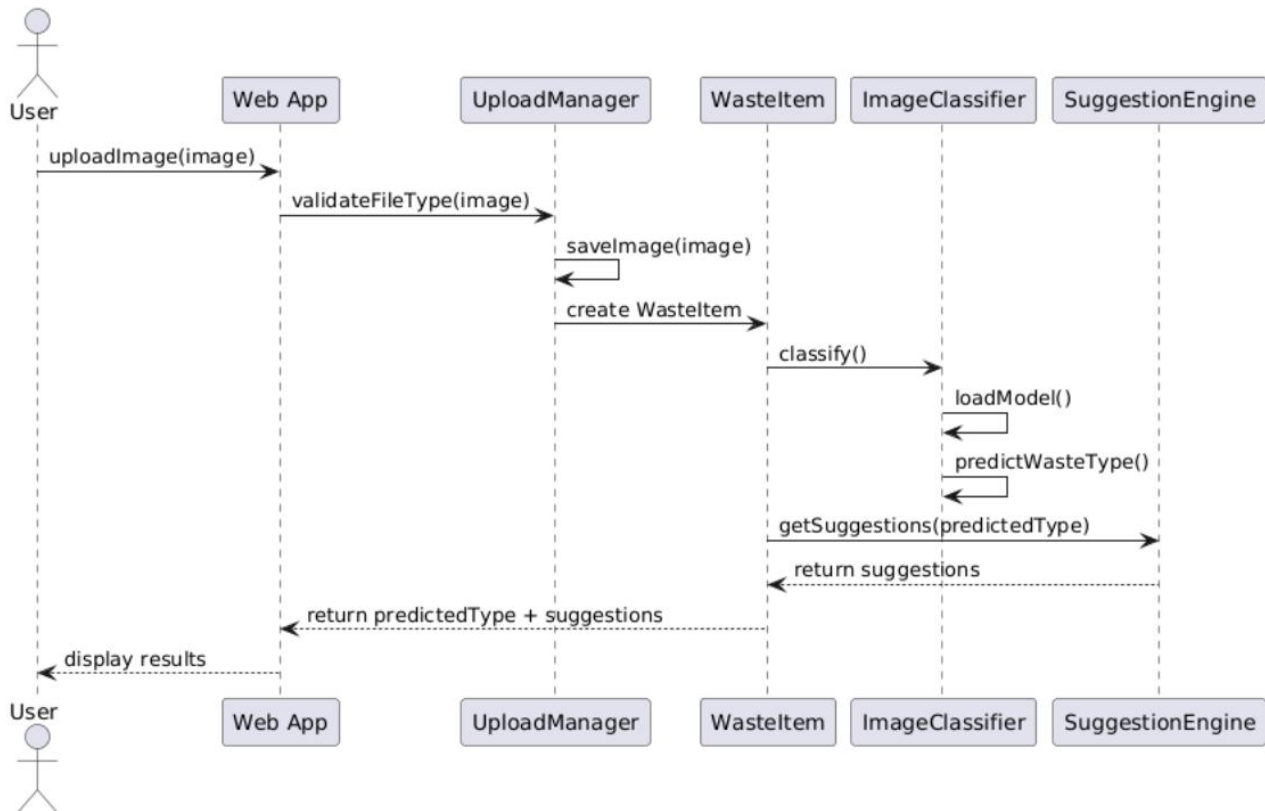


Class Diagram:

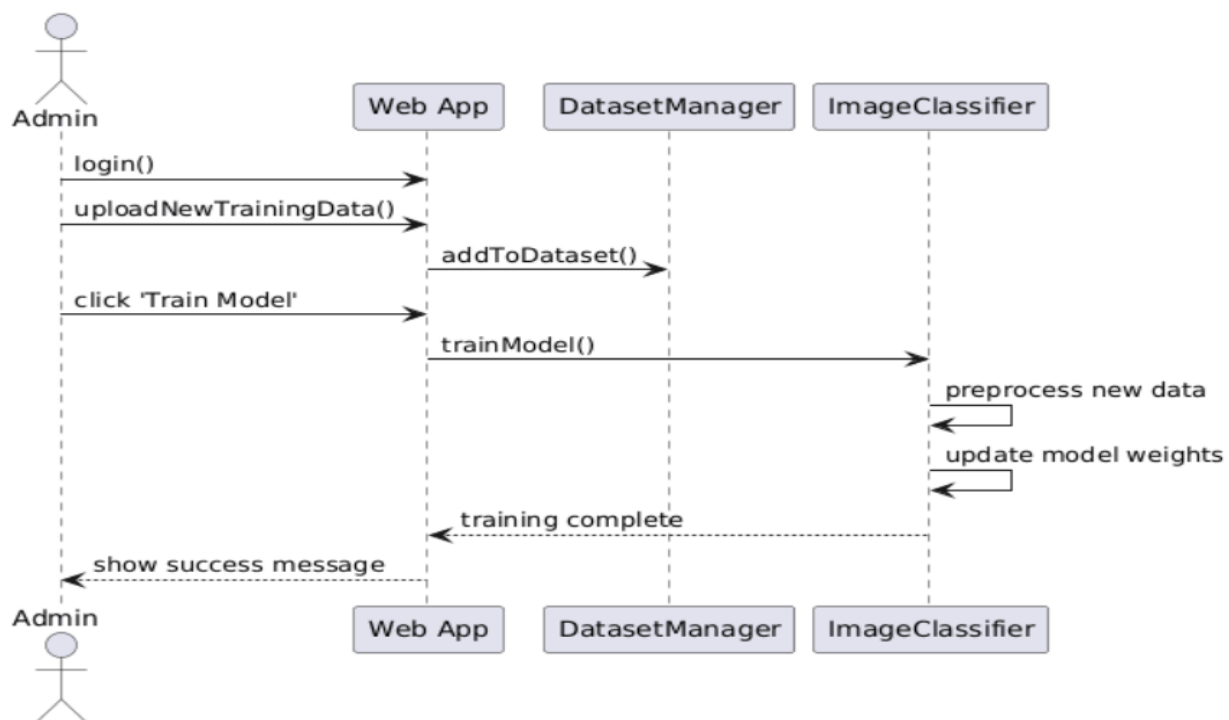


Sequence Diagram:

Scenario 1: Waste Image Upload & Classification



Scenario 2: Admin Trains the Model and Updates Dataset



Complete Component Diagram:

